AI ALGORITHM FLOW DOCUMENTATION

Face Recognition Payment System

Algorithm pipeline from face recording to payment verification

OpenCV Haar Cascade | DeepFace MiniFASNet | Facenet · 128-d Embedding | KNN · Cosine Distance | FastAPI · Python

---

1 Face Enrollment (Recording)

User registers their face biometric data — runs once per user

INPUT **Browser Captures 20 Frames**

Web camera accessed via `navigator.mediaDevices.getUserMedia`. JavaScript captures 20 JPEG frames from the live video stream and POSTs them as `multipart/form-data` to Spring Boot.

GATEWAY **Spring Boot → Python FastAPI**

`POST /api/face/enroll` (JWT-authenticated) receives the image files. FaceService forwards them to the Python AI engine at `http://face-service:8000/enroll` via `RestTemplate`.

ALGORITHM 1 **Face Detection — Haar Cascade**

Each frame is decoded into a BGR image with OpenCV. Detection runs on each frame individually.

VIOLA-JONES / HAAR CASCADE ALGORITHM

1. Convert BGR → **Grayscale**
2. Apply `haarcascade_frontalface_alt.xml` classifier (via DeepFace detector)
3. `detectMultiScale(scaleFactor=1.1, minNeighbors=3, minSize=(50,50))`
4. Select **largest face** by bounding box area (w × h)
5. Align face → pass to Facenet for embedding

ALGORITHM 2 **Feature Extraction — Facenet CNN Embedding**

Each detected face is passed through the **Facenet** deep CNN to produce an identity-discriminative embedding. Trained on millions of faces, Facenet maps each face into a 128-dimensional vector where

same-identity faces are close together and different identities are far apart in the embedding space.

FACENET EMBEDDING (DEEPFACE)

```
DeepFace.represent(img_path=frame, model_name='Facenet', align=True)
```

Model: Facenet — 22-layer CNN trained with triplet loss
Output: 128-dimensional L2-normalized embedding

Result: matrix of shape **(N samples × 128 features)**

VALIDATION

Minimum 3 Valid Samples Required

If fewer than 3 face embeddings produced → HTTP 400 error (poor lighting / position). Since Facenet embeddings are highly discriminative, only 3 samples suffice (vs. the larger sample requirement of raw-pixel methods).

STORAGE

Save Embeddings to Disk

```
np.save("face_dataset/{userId}.npy", embeddings)
```

Overwrites any previous enrollment for the same user (re-enrollment is supported). File contains the **(N × 128)** matrix of Facenet embedding vectors.

INPUT FRAMES	MIN REQUIRED	EMBEDDING MODEL	EMBEDDING DIMS
20 captured per enrollment	**3** valid embeddings	**Facenet** trained CNN encoder	**128** per face sample

2 Face Verification (Checkout / Payment)

Two-stage pipeline: Anti-Spoofing → Identity Matching

INPUT **Browser Captures 1 Frame (Checkout)**

During store checkout, user opens Face Verification modal. A single JPEG frame is captured from the live webcam. Sent as `multipart/form-data` to Spring Boot alongside the cart items.

GATEWAY **Spring Boot → Python FastAPI**

`POST /api/store/orders/checkout` (JWT-authenticated). OrderService calls `FaceService.verifyFace(userId, faceImage)` → forwards to `http://face-service:8000/verify`.

STAGE 1 **Liveness Detection — Anti-Spoofing**

Before any identity check, the system verifies the face is from a real, live person — not a photo, screen replay, or 3D mask.

DEEPFACE MINIFASNET (PASSIVE LIVENESS)

```
DeepFace.extract_faces(img_path=frame, anti_spoofing=True, enforce_detection=False)
```

Model: MiniFASNet — lightweight CNN trained on spoofing datasets
Output:

- `is_real` → *boolean* (True = live person)
- `antispoof_score` → *float 0.0 – 1.0* (confidence)

✓ is_real = True

Live person detected → proceed to Stage 2 identity matching

✗ is_real = False

Spoof detected (photo / screen / mask) → return `{ match: false, reason: "liveness_failed" }`
→ **Payment DENIED**

STAGE 2 **Probe Face Detection & Embedding**

Same pipeline as enrollment is applied to the verification frame: detect, align, then encode with Facenet.

HAAR CASCADE + FACENET (SAME AS ENROLLMENT)

BGR → Grayscale → `detectMultiScale(1.1, 3, minSize=(50,50))` → largest face
Align face → Facenet CNN → **128-dim probe embedding**

STAGE 2 **Identity Matching — KNN with Cosine Distance**

The probe embedding is compared against all enrolled embeddings of the claimed user using **cosine distance** (angle between vectors) instead of Euclidean — more discriminative on L2-normalized

Facenet embeddings.

K-NEAREST NEIGHBORS (KNN) ON FACENET EMBEDDINGS

1. Load enrolled embeddings
`enrolled = np.load("face_dataset/{userId}.npy")` → shape (N × 128)

2. Compute cosine distance to every enrolled embedding:

```
d(v_probe, v_i) = 1 - ( v_probe · v_i ) / ( ||v_probe|| · ||v_i|| )
```

3. Sort distances ascending, take top-5 (k=5)
`distances = sorted(distances)[:5]`

4. Average the 5 nearest cosine distances:
`avg_dist = mean(top-5 distances)`

5. Threshold decision:
`match = (avg_dist < 0.40)` — DeepFace's recommended Facenet threshold

✓ avg_dist < 0.40
Embeddings match (typically 0.05–0.30 for same person) →
return `{ match: true }`
→ **Payment APPROVED**
Balance deducted, order created

✗ avg_dist ≥ 0.40
Embeddings don't match (typically 0.50–0.80 for different person) →
return `{ match: false }`
→ **Payment DENIED**

ANTI-SPOOF MODEL
MiniFAS
DeepFace passive liveness

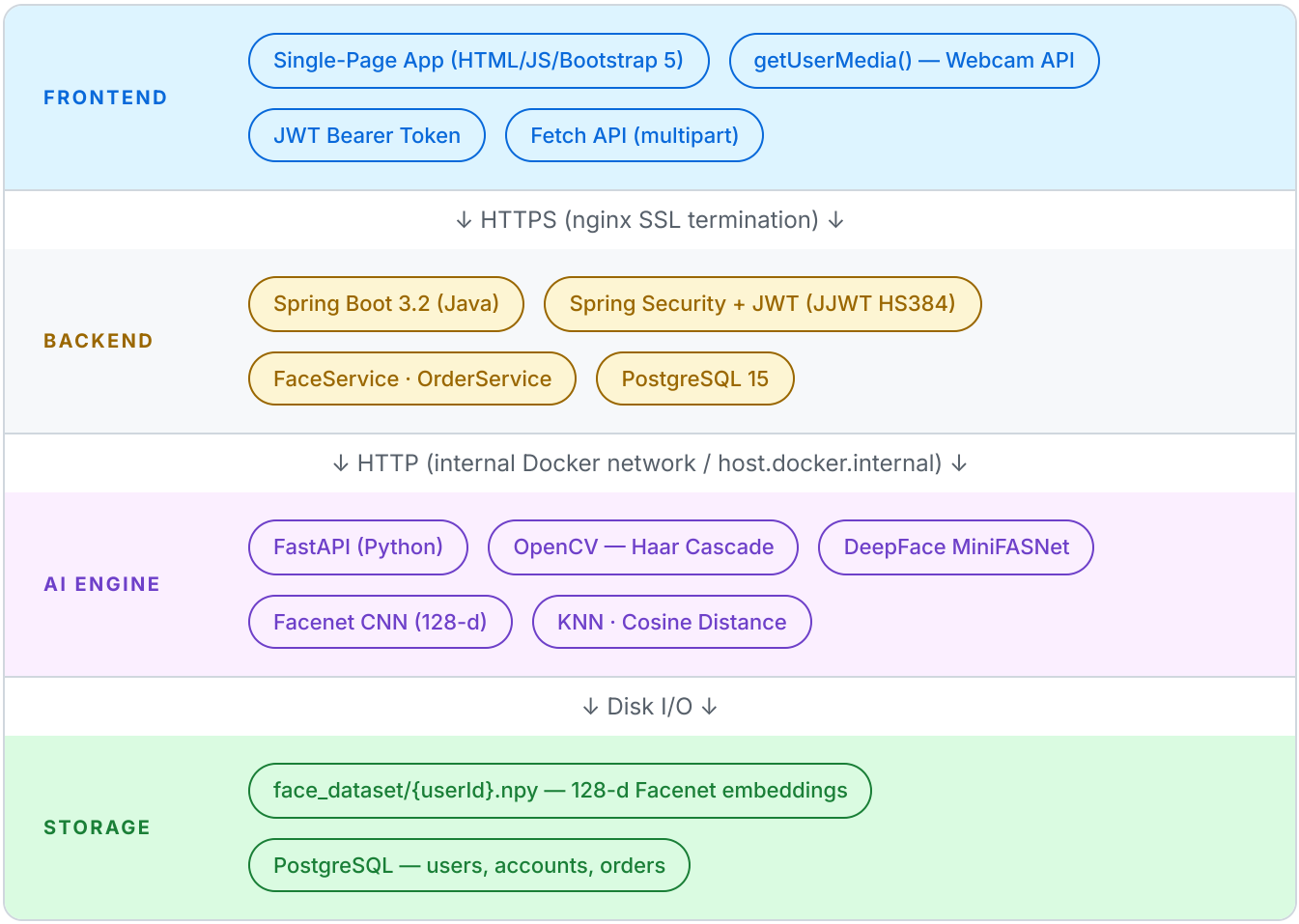
KNN — K VALUE
5
nearest neighbors

MATCH THRESHOLD
<0.40
avg cosine distance

EMBEDDING SPACE
128
Facenet dimensions

3 System Architecture Overview

End-to-end flow from browser to AI engine and back



4 Algorithm Comparison & Design Choices

Why these algorithms were selected for this system

ALGORITHM	CATEGORY	USED IN	ADVANTAGE	TRADE-OFF
Haar Cascade `haarcascade_frontalface_alt.xml`	Face Detection	Enroll + Verify	Real-time speed, no GPU, OpenCV built-in	Frontal face only, sensitive to rotation
Facenet (CNN) `DeepFace.represent(model='Facenet')`	Feature Embedding	Enroll + Verify	Identity-discriminative 128-d embeddings, robust to lighting	Heavier than raw pixels, requires model weights (~92 MB)

ALGORITHM	CATEGORY	USED IN	ADVANTAGE	TRADE-OFF
KNN + Cosine Distance `k=5, threshold=0.40`	**Identity Matching**	**Verify**	**Wide same/different separation on embeddings, no training**	**Linear search O(N·d) — fine here since N is small (~20)**
DeepFace MiniFASNet	**Anti-Spoofing**	**Verify**	**Detects photo/screen/mask attacks passively**	**Requires DeepFace library, slightly slower**
Raw Pixel Flattening *(initial approach)*	Feature Extraction	— replaced —	Simple, no model needed	Suffered false-accepts: different faces < threshold
ArcFace / VGG-Face	Deep Embedding (alternative)	—	Higher accuracy than Facenet (~99.6% on LFW)	Larger model, slower inference